

Projeto Técnico — Sistema de Emissão de Notas Fiscais

Detalhamento técnico da solução — Desafio Korp ERP

Repositório: github.com/vidalclean/Korp_Testes_MariaCleane

1. Visão geral

O sistema foi desenvolvido seguindo o escopo do desafio técnico da Korp: cadastro de produtos, cadastro e impressão de notas fiscais, com baixa automática de estoque. A solução foi estruturada como dois microsserviços independentes em Go (Estoque e Faturamento), cada um com seu próprio banco de dados PostgreSQL, e um frontend em Angular que consome os dois serviços diretamente.

2. Arquitetura de microsserviços

A solução é composta por três partes independentes, que rodam em processos separados:

- Serviço de Estoque (Go, porta 8081) — responsável pelo cadastro de produtos e controle de saldo. Possui seu próprio banco de dados (db_estoque).
- Serviço de Faturamento (Go, porta 8082) — responsável pelo cadastro e impressão de notas fiscais. Possui seu próprio banco de dados (db_faturamento) e se comunica com o Serviço de Estoque via HTTP/REST.
- Frontend (Angular, porta 4200) — interface visual que consome os dois serviços diretamente via HTTP.

Essa separação garante que cada serviço possa ser desenvolvido, testado, implantado e escalado de forma independente, e que uma falha em um serviço não derrube o outro — apenas limite as operações que dependem dele (ver seção 6).

3. Telas desenvolvidas

- Tela de Produtos: formulário de cadastro (código, descrição, saldo inicial) e tabela com a lista de produtos cadastrados, atualizada em tempo real após cada cadastro.
- Tela de Notas Fiscais: formulário para montar uma nota (seleção de produto e quantidade, com possibilidade de adicionar múltiplos itens antes de criar a nota) e tabela com todas as notas, mostrando número, status, itens e um botão de impressão.

4. Funcionalidades implementadas

- Cadastro de produtos com código, descrição e saldo.
- Cadastro de notas fiscais com numeração sequencial automática, status inicial "Aberta" e múltiplos produtos com suas respectivas quantidades.
- Impressão de nota fiscal: ao clicar em "Imprimir", é exibido um indicador de processamento (spinner); ao concluir, o status da nota muda para "Fechada" e o saldo dos produtos envolvidos é atualizado automaticamente.
- Bloqueio de impressão para notas que não estão com status "Aberta" (botão desabilitado na interface e validação também no backend).
- Persistência real dos dados em PostgreSQL (nenhum dado é mantido apenas em memória).

5. Detalhamento técnico — Frontend (Angular)

Ciclos de vida do Angular utilizados

Foi utilizado o hook `ngOnInit()` nos componentes `Produtos` e `Notas`. Esse ciclo de vida roda uma única vez, logo após o Angular montar o componente na tela, sendo o momento correto para disparar as chamadas iniciais de carregamento de dados (`carregarProdutos()` e `carregarNotas()`), evitando fazer essas chamadas dentro do construtor.

Uso da biblioteca RxJS

RxJS foi utilizado de duas formas principais:

- `BehaviorSubject`: cada serviço (`ProdutoService` e `NotaService`) mantém um `BehaviorSubject` com a lista mais atual de produtos/notas. Isso permite que qualquer componente "assine" (`subscribe`) esse estado e seja notificado automaticamente sempre que a lista mudar, sem precisar recarregar a página manualmente.
- `Operator tap`: usado para, após criar um produto/nota ou imprimir uma nota com sucesso, recarregar automaticamente a lista mais atualizada do backend.
- `Async pipe` (`| async`): usado diretamente nos templates HTML para consumir os `Observables` sem necessidade de `subscribe/unsubscribe` manual, evitando vazamento de memória (`memory leaks`).

Outras bibliotecas utilizadas

- `@angular/forms` (Reactive Forms) — utilizado para os formulários de cadastro de produto e criação de nota, com validações (campos obrigatórios, valores mínimos).
- `@angular/common/http` (`HttpClient`) — utilizado para toda a comunicação HTTP com os dois microsserviços.

Bibliotecas para componentes visuais

Foi utilizado o Angular Material para os componentes visuais da interface: `mat-toolbar` (barra de navegação), `mat-card`, `mat-form-field` / `mat-input` / `mat-select` (formulários), `mat-table` (tabelas), `mat-button`, `mat-icon`, `mat-progress-spinner` (indicador de processamento durante a impressão da nota) e `mat-snack-bar` (mensagens de feedback ao usuário, como sucesso ou erro).

6. Detalhamento técnico — Backend (Go)

Gerenciamento de dependências no Golang

O gerenciamento de dependências foi feito com `Go Modules`, o sistema nativo do Go. Cada microsserviço possui seu próprio arquivo `go.mod` (definindo o módulo e suas dependências diretas) e `go.sum` (com o hash de verificação de cada dependência, garantindo integridade). As dependências foram obtidas com o comando `"go get"`, sem necessidade de gerenciador externo.

Frameworks / bibliotecas utilizadas no Golang

- `go-chi/chi` — roteador HTTP leve, utilizado para organizar as rotas de cada serviço e extrair parâmetros de URL (ex: `id` do produto, número da nota).
- `lib/pq` — driver PostgreSQL para Go, utilizado através do pacote nativo `database/sql` para todas as operações de banco (`SELECT`, `INSERT`, `UPDATE`, transações).

Optou-se por não usar um framework web completo (como `Gin` ou `Echo`) nem um ORM, mantendo o backend enxuto e utilizando SQL diretamente — o que facilita o controle fino sobre transações, essencial para o tratamento de concorrência descrito abaixo.

Tratamento de erros e exceções no backend

O Go não possui exceções como outras linguagens; erros são valores retornados explicitamente pelas funções e verificados em cada chamada. A aplicação segue esse padrão:

- Toda função que pode falhar (consulta ao banco, decodificação de JSON, chamada HTTP) retorna um valor de erro, verificado imediatamente com `"if err != nil"`.
- Erros são convertidos em respostas HTTP claras para o cliente, com código de status apropriado (400 para dados inválidos, 404 para recurso não encontrado, 409 para conflito de estado, 500 para erros internos, 502 quando um serviço dependente está indisponível) e uma mensagem JSON explicando o problema.

- No cliente HTTP do Faturamento em direção ao Estoque, foi configurado um timeout (5 segundos) para que uma falha de rede ou indisponibilidade do Estoque nunca deixe o Faturamento travado esperando indefinidamente.

7. Tratamento de falhas entre microserviços

Cenário testado: o Serviço de Estoque é derrubado propositalmente enquanto o Serviço de Faturamento está no ar.

- Ao tentar imprimir uma nota fiscal, o Faturamento tenta chamar o Estoque via HTTP e recebe um erro de conexão (ou timeout).
- Esse erro é capturado e transformado em uma resposta clara para o usuário (HTTP 502, com a mensagem explicando que o serviço de estoque está indisponível).
- A nota permanece com status "Aberta" — nada é corrompido ou perdido — permitindo que o usuário tente novamente mais tarde.
- Ao religar o Estoque, a mesma nota pode ser impressa normalmente, confirmando a recuperação do sistema após a falha.

No frontend, esse erro aparece como uma mensagem (snackbar) com o texto retornado pelo backend, dando feedback imediato e compreensível ao usuário.

8. Conexão com banco de dados

Cada microserviço se conecta a um banco PostgreSQL próprio (db_estoque e db_faturamento), com as tabelas criadas automaticamente na inicialização do serviço (migração simples via SQL, executada no startup). Toda a persistência é real — não há dados mantidos apenas em memória.

9. Requisito opcional implementado — Tratamento de concorrência

Foi implementado o tratamento de concorrência no Serviço de Estoque: quando dois pedidos de baixa de estoque chegam ao mesmo tempo para o mesmo produto, a operação é protegida por uma transação SQL com `SELECT ... FOR UPDATE`, que trava a linha do produto no banco de dados até a transação ser concluída. Isso garante que, mesmo em um cenário como "produto com saldo 1 sendo utilizado simultaneamente por duas notas", apenas uma das operações terá sucesso e a outra receberá o erro de saldo insuficiente, evitando saldo negativo ou inconsistências.

10. Rotas dos serviços

Serviço de Estoque (localhost:8081)

Método	Rota	Descrição
POST	/produtos	Cadastra um novo produto
GET	/produtos	Lista todos os produtos
POST	/produtos/{id}/baixa	Dá baixa em uma quantidade do saldo

Serviço de Faturamento (localhost:8082)

Método	Rota	Descrição
POST	/notas	Cria uma nova nota fiscal (status Aberta)
GET	/notas	Lista todas as notas fiscais
POST	/notas/{numero}/imprimir	Imprime a nota: baixa o estoque e fecha a nota

11. Como executar o projeto

- Subir o banco de dados PostgreSQL (via `docker-compose up -d`, ou instalação local — ver README).
- Rodar o Serviço de Estoque: `cd estoque-service && go run .`

- Rodar o Serviço de Faturamento: `cd faturamento-service && go run .`
- Rodar o Frontend: `cd frontend && ng serve`
- Acessar `http://localhost:4200` no navegador.

12. Sobre o uso de C# e LINQ

Não aplicável — a implementação do backend foi feita inteiramente em Go, conforme detalhado nas seções acima.